Gen AI - Unit 3 Assignment: Building a Production Advanced RAG System

Name	Diya D Bhat
SRN	PES2UG23CS183
Section	C
Date	31/03/26

Part 1 - Document Corpus Setup

```
Corpus size: 15 documents

  doc_00: Transformers use self-attention mechanisms to process sequences in parallel, rep
  doc_01: BERT is a bidirectional encoder pre-trained using masked language modelling and
  doc_02: The BM25 algorithm ranks documents using term frequency saturation and inverse d
  doc_03: Gradient descent minimizes the loss function by iteratively updating weights in
  doc_04: Neural networks learn by propagating the error signal backwards through layers u
  doc_05: Proper weight initialization prevents vanishing and exploding gradients, enablin
  doc_06: Batch normalization normalizes layer inputs during training, accelerating conver
  doc_07: Retrieval Augmented Generation grounds language model responses by injecting ret
  doc_08: LoRA fine-tunes large language models by injecting trainable low-rank matrices i
  doc_09: GPTQ quantizes model weights to INT4 post-training by minimizing layer-wise reco
  doc_10: Mixture of Experts models activate only a sparse subset of expert networks per t
  doc_11: AdamW decouples weight decay from the gradient update in the Adam optimizer, imp
  doc_12: Cosine similarity measures the angle between two vectors and is used to compare
  doc_13: Dropout randomly deactivates neurons during training to prevent co-adaptation an
  doc_14: Cross-entropy loss measures the divergence between predicted probability distrib
```

Part 2 - Implement Hybrid Retrieval

```
WARNING:tensorflow:From C:\Users\diyab\AppData\Local\Programs\Python\Python313\Lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_s
oftmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

HybridRetriever built.

Query: 'How do neural networks learn?'
  doc_id   RRF score    BM25 rank    SBERT rank   text
  ------------------------------------------------------------------------------
  doc_4    0.032787   1            1            Neural networks learn by propagating the error signal b
  doc_10   0.032002   3            2            Mixture of Experts models activate only a sparse subset
  doc_0    0.031054   2            7            Transformers use self-attention mechanisms to process s

Query: 'BM25 term frequency ranking'
  doc_id   RRF score    BM25 rank    SBERT rank   text
  ------------------------------------------------------------------------------
  doc_2    0.032787   1            1            The BM25 algorithm ranks documents using term frequency
  doc_14   0.031258   3            5            Cross-entropy loss measures the divergence between pred
  doc_8    0.030835   8            2            LoRA fine-tunes large language models by injecting trai

Query: 'AdamW optimizer weight decay'
  doc_id   RRF score    BM25 rank    SBERT rank   text
  ------------------------------------------------------------------------------
  doc_11   0.032787   1            1            AdamW decouples weight decay from the gradient update i
  doc_5    0.032258   2            2            Proper weight initialization prevents vanishing and exp
  doc_8    0.030579   3            8            LoRA fine-tunes large language models by injecting trai
```

Part 3 - Implement a Cross-Encoder Re-Ranker

```
Cross-encoder loaded: ms-marco-MiniLM-L-6-v2

Query: 'How does attention work in language models?'

Before Re-Ranking (Hybrid order):
  #1  [RRF=0.03279] LoRA fine-tunes large language models by injecting trainable low-rank matrices into frozen attention weight projections.
  #2  [RRF=0.03175] BERT is a bidirectional encoder pre-trained using masked language modelling and next sentence prediction.
  #3  [RRF=0.03175] Retrieval Augmented Generation grounds language model responses by injecting retrieved documents into the prompt context.
  #4  [RRF=0.03126] Transformers use self-attention mechanisms to process sequences in parallel, replacing recurrent networks entirely.
  #5  [RRF=0.03016] AdamW decouples weight decay from the gradient update in the Adam optimizer, improving regularization in transformer training.

After Cross-Encoder Re-Ranking (top 3):
  #1  [CE score=2.2710] LoRA fine-tunes large language models by injecting trainable low-rank matrices into frozen attention weight projections.
  #2  [CE score=-5.9369] Transformers use self-attention mechanisms to process sequences in parallel, replacing recurrent networks entirely.
  #3  [CE score=-6.1499] Retrieval Augmented Generation grounds language model responses by injecting retrieved documents into the prompt context.
```

Part 4 - Implement Query Expansion

```
Original query: 'How do transformers encode meaning?'

[HyDE] Hypothetical Document:
  Transformers encode meaning through a self-attention mechanism, which allows the model to weigh the importance of different input elements relative to
one another. This is achieved by computing attention weights, which are calculated as the dot product of the query and key vectors, normalized by the squ
are root of the key vector's dimensionality. The resulting attention weights are then used to compute a weighted sum of the value vectors, effectively se
lecting and combining relevant information from the input sequence to produce a contextualized representation. This process enables the transformer to ca
pture complex relationships and dependencies within the input data, facilitating the encoding of nuanced meaning and context.

[Multi-Query] Variants:
  (original): How do transformers encode meaning?
  (variant 1): What mechanisms do transformers use to represent semantic meaning in text data?
  (variant 2): How do transformer models capture and encode the underlying meaning of input text?
```

Part 5 - End-to-End Pipeline

```
Query: 'how do transformers encode meaning?'
======================================================================
[1] HyDE Expansion:
    Transformers encode meaning through a self-attention mechanism, which allows the model to weigh the importance of different input elements relative t
o one another. This is achieved by computing attent...

[2] Hybrid Retrieval (top 5):
    #1 [RRF=0.03252 | BM25_rank=2 | SBERT_rank=1] Transformers use self-attention mechanisms to process sequences in par
    #2 [RRF=0.03154 | BM25_rank=1 | SBERT_rank=6] Cosine similarity measures the angle between two vectors and is used t
    #3 [RRF=0.03083 | BM25_rank=8 | SBERT_rank=2] AdamW decouples weight decay from the gradient update in the Adam opti
    #4 [RRF=0.03037 | BM25_rank=9 | SBERT_rank=3] LoRA fine-tunes large language models by injecting trainable low-rank
    #5 [RRF=0.02988 | BM25_rank=5 | SBERT_rank=9] Gradient descent minimizes the loss function by iteratively updating w

[3] After Re-Ranking (top 3):
    #1 [CE=-2.6834] Transformers use self-attention mechanisms to process sequences in parallel, replacing recurrent networks entirely.
    #2 [CE=-11.1625] Cosine similarity measures the angle between two vectors and is used to compare query and document embeddings in dense retrieval.
    #3 [CE=-11.1868] AdamW decouples weight decay from the gradient update in the Adam optimizer, improving regularization in transformer training.

[4] Final Answer:
    I don't have enough information to answer this.


    Query: 'optimization techniques for training'
    ======================================================================
    [1] HyDE Expansion:
        During the training process of machine learning models, several optimization techniques can be employed to improve convergence rates and reduce compu
    tational costs. One such technique is Stochastic Gr...

    [2] Hybrid Retrieval (top 5):
        #1 [RRF=0.03279 | BM25_rank=1 | SBERT_rank=1] Gradient descent minimizes the loss function by iteratively updating w
        #2 [RRF=0.03200 | BM25_rank=3 | SBERT_rank=2] Batch normalization normalizes layer inputs during training, accelerat
        #3 [RRF=0.03105 | BM25_rank=2 | SBERT_rank=7] Dropout randomly deactivates neurons during training to prevent co-ada
        #4 [RRF=0.03102 | BM25_rank=6 | SBERT_rank=3] AdamW decouples weight decay from the gradient update in the Adam opti
        #5 [RRF=0.03033 | BM25_rank=4 | SBERT_rank=8] GPTQ quantizes model weights to INT4 post-training by minimizing layer

    [3] After Re-Ranking (top 3):
        #1 [CE=-3.6821] AdamW decouples weight decay from the gradient update in the Adam optimizer, improving regularization in transformer training.
        #2 [CE=-5.0406] Batch normalization normalizes layer inputs during training, accelerating convergence and acting as a regularizer.
        #3 [CE=-6.2086] GPTQ quantizes model weights to INT4 post-training by minimizing layer-wise reconstruction error on a calibration set.

    [4] Final Answer:
        Based on the provided context, the following optimization techniques for training are mentioned:

    1. Adam optimizer with weight decay (decoupled) - as mentioned in Document 1.
    2. Batch normalization - as mentioned in Document 2, which accelerates convergence and acts as a regularizer.
    3. GPTQ - a technique for quantizing model weights post-training, as mentioned in Document 3.
```

```
Query: 'what is the role of the router in mixture of experts?'
================================================================
[1] HyDE Expansion:
    In the Mixture of Experts (MoE) architecture, the router plays a crucial role in determining the contribution of each expert network to the final out
put. The router is typically implemented as a softm...

[2] Hybrid Retrieval (top 5):
    #1 [RRF=0.03279 | BM25_rank=1 | SBERT_rank=1] Mixture of Experts models activate only a sparse subset of expert netw
    #2 [RRF=0.03125 | BM25_rank=4 | SBERT_rank=4] AdamW decouples weight decay from the gradient update in the Adam opti
    #3 [RRF=0.03105 | BM25_rank=7 | SBERT_rank=2] Batch normalization normalizes layer inputs during training, accelerat
    #4 [RRF=0.03105 | BM25_rank=2 | SBERT_rank=7] GPTQ quantizes model weights to INT4 post-training by minimizing layer
    #5 [RRF=0.03054 | BM25_rank=5 | SBERT_rank=6] Gradient descent minimizes the loss function by iteratively updating w

[3] After Re-Ranking (top 3):
    #1 [CE=5.2716] Mixture of Experts models activate only a sparse subset of expert networks per token, controlled by a learned router.
    #2 [CE=-10.6989] Batch normalization normalizes layer inputs during training, accelerating convergence and acting as a regularizer.
    #3 [CE=-11.1695] AdamW decouples weight decay from the gradient update in the Adam optimizer, improving regularization in transformer training.

[4] Final Answer:
    The router in Mixture of Experts models controls which expert networks are activated per token, and it does so in a sparse manner.
```

Part 6 - Comparison Experiment

```
sbert_model = HybridRetriever(corpus).sbert
doc_vecs_naive = sbert_model.encode(corpus, convert_to_numpy=True)
doc_vecs_naive = doc_vecs_naive / np.linalg.norm(doc_vecs_naive, axis=1, keepdims=True)

def naive_rag(user_query: str) -> str:
    q_vec   = sbert_model.encode([user_query], convert_to_numpy=True)[0]
    q_vec   = q_vec / np.linalg.norm(q_vec)
    scores  = doc_vecs_naive @ q_vec
    top_idx = int(np.argsort(scores)[::-1][0])
    return corpus[top_idx]

comparison_queries = [
    "how do transformers encode meaning?",
    "optimization techniques for training",
    "what is the role of the router in mixture of experts?",
]

print("Running comparison experiment...")
print()

naive_tops = [naive_rag(q) for q in comparison_queries]
advanced_tops = []

for q in comparison_queries:
    expanded   = expand_hyde(q)
    candidates = hybrid.retrieve(expanded, top_k=5)
    top_docs   = rerank(q, candidates, top_k=3)
    advanced_tops.append(top_docs[0]["text"])

print("Done.")
```

```
Running comparison experiment...

Done.
```

Comparison Table and Observations

Comparison Table

Query	Naïve RAG Top Doc	Advanced RAG Top Doc	Different?
"how do transformers encode meaning?"	Transformers use self-attention mechanisms to process sequences in parallel, replacing recurrent networks entirely.	Transformers use self-attention mechanisms to process sequences in parallel, replacing recurrent networks entirely.	No — both find the same top doc for this semantic query (dense retrieval already works well here)
"optimization techniques for training"	Gradient descent minimizes the loss function by iteratively updating weights in the direction of the negative gradient.	AdamW decouples weight decay from the gradient update in the Adam optimizer, improving regularization in transformer training.	**Yes** — Advanced RAG surfaces the AdamW doc (a more specific optimization technique) via HyDE expansion + re-ranking
"what is the role of the router in mixture of experts?"	Mixture of Experts models activate only a sparse subset of expert networks per token, controlled by a learned router.	Mixture of Experts models activate only a sparse subset of expert networks per token, controlled by a learned router.	No — the MoE doc is semantically distinctive enough that both pipelines find it

Observations

- **Query 1** (semantic): Naïve RAG performs as well as Advanced RAG because "transformers encode meaning" is semantically close to doc_0 already. Dense retrieval handles this well without expansion.
- **Query 2** (keyword/optimization): Advanced RAG wins — the HyDE expansion generates text mentioning "AdamW", "learning rate", "weight decay", pulling up a more precise document. Naïve RAG stops at the generic gradient descent doc.
- **Query 3** (technical): Both find the correct document. The MoE concept is distinctive enough in vector space that no expansion is needed.

Key takeaway: HyDE + Re-Ranking provides the biggest gains on short, ambiguous queries where the user vocabulary differs from document vocabulary — exactly the vocabulary gap this unit is designed to solve.

Bonus Challenges:

Bonus 1 - Weighted RRF

```
Query: 'BM25 term frequency ranking'
  alpha    Top-1 doc (first 70 chars)
  ------------------------------------------------------------------------
  alpha=0.3  The BM25 algorithm ranks documents using term frequency saturation and
  alpha=0.5  The BM25 algorithm ranks documents using term frequency saturation and
  alpha=0.7  The BM25 algorithm ranks documents using term frequency saturation and

Query: 'how do neural networks learn from examples?'
  alpha    Top-1 doc (first 70 chars)
  ------------------------------------------------------------------------
  alpha=0.3  Neural networks learn by propagating the error signal backwards throug
  alpha=0.5  Neural networks learn by propagating the error signal backwards throug
  alpha=0.7  Neural networks learn by propagating the error signal backwards throug

Interpretation:
  Keyword query  → higher alpha (more BM25 weight) surfaces the exact-match doc faster.
  Semantic query → lower alpha (more SBERT weight) surfaces the semantically closest doc.
```

Bonus 2 - Chunk Size Study

```
Original document: 180 words
Query: 'How does self-attention capture long-range dependencies?'

Chunk size = 50 words → 4 chunk(s)
  Best CE score: 5.7870
  Best chunk:    long-range dependencies without recurrence. The architecture consists of an encoder and a decoder, each made of stacked ...

Chunk size = 100 words → 2 chunk(s)
  Best CE score: 7.5497
  Best chunk:    The Transformer architecture was introduced in the paper Attention Is All You Need by Vaswani et al. in 2017. It replace...

Chunk size = 200 words → 1 chunk(s)
  Best CE score: 6.9631
  Best chunk:    The Transformer architecture was introduced in the paper Attention Is All You Need by Vaswani et al. in 2017. It replace...

Interpretation:
  Smaller chunks isolate the relevant sentence → higher cross-encoder score.
  Larger chunks dilute the signal with unrelated sentences → lower score.
  Optimal chunk size balances precision (small) vs context richness (large).
```

Bonus 3 - Add ColBERT

```
Triple Hybrid (BM25 + SBERT + ColBERT) Retrieval

Query: 'how does attention capture meaning?'
  doc_id   RRF          BM25_r    SBERT_r    ColBERT_r    text
  ------------------------------------------------------------------------------------
  doc_8    0.048660   1         3          1            LoRA fine-tunes large language models by inje
  doc_7    0.046708   8         2          3            Retrieval Augmented Generation grounds langua
  doc_12   0.045935   4         5          7            Cosine similarity measures the angle between

Query: 'BM25 term frequency ranking'
  doc_id   RRF          BM25_r    SBERT_r    ColBERT_r    text
  ------------------------------------------------------------------------------------
  doc_2    0.049180   1         1          1            The BM25 algorithm ranks documents using term
  doc_8    0.046964   8         2          2            LoRA fine-tunes large language models by inje
  doc_14   0.046883   3         5          4            Cross-entropy loss measures the divergence be

Query: 'AdamW weight decay optimizer'
  doc_id   RRF          BM25_r    SBERT_r    ColBERT_r    text
  ------------------------------------------------------------------------------------
  doc_11   0.049180   1         1          1            AdamW decouples weight decay from the gradien
  doc_5    0.048387   2         2          2            Proper weight initialization prevents vanishi
  doc_8    0.046183   3         7          5            LoRA fine-tunes large language models by inje

Observation: ColBERT as a 3rd retriever provides additional signal from token-level
late interaction, particularly helping on queries with rare token overlap.
```